

第十一章

目标代码生成

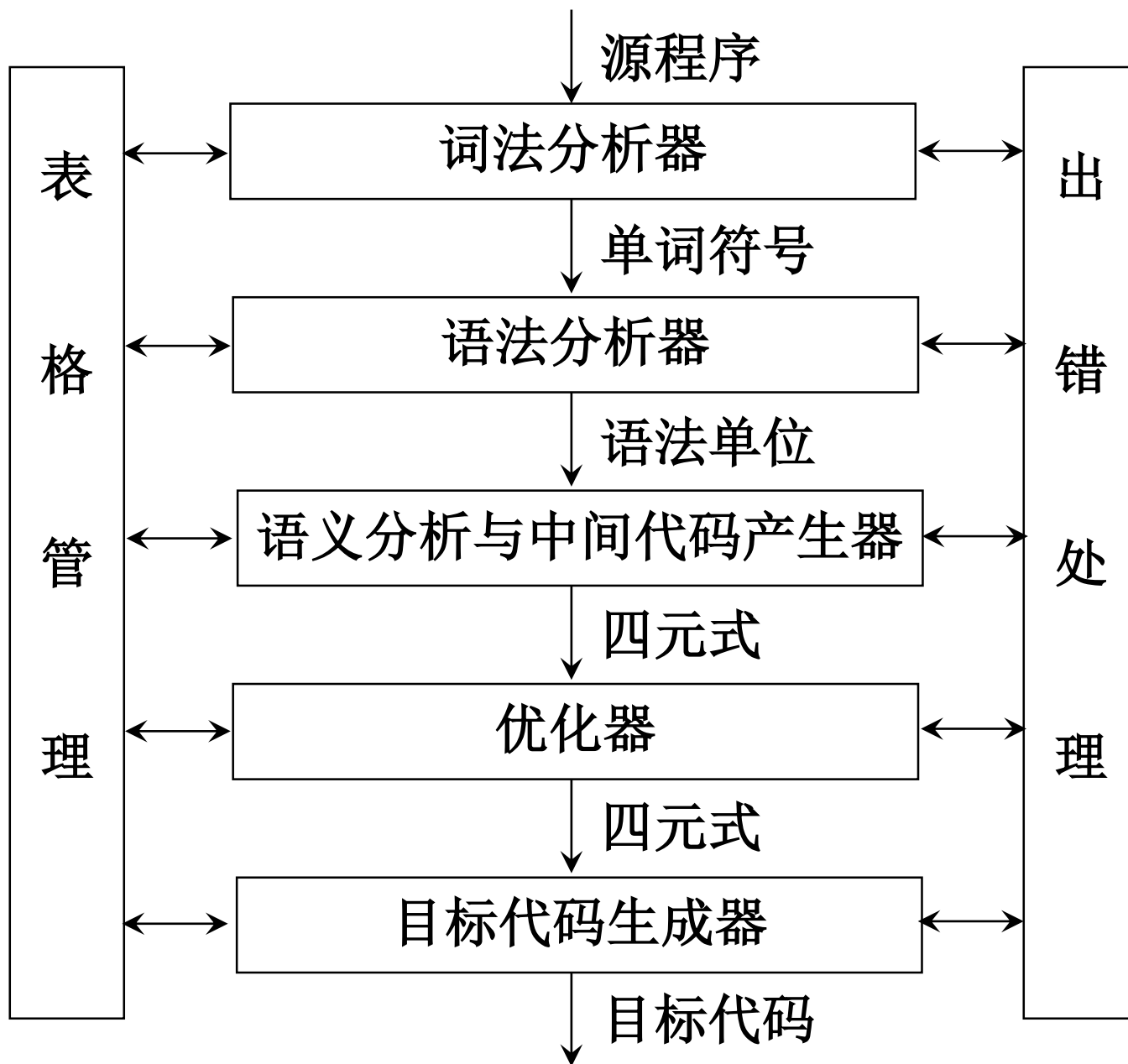
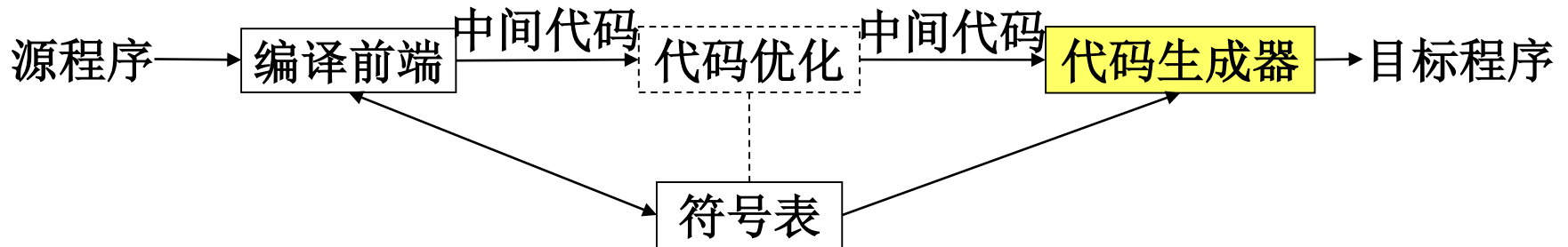


图1.1 编译程序总框

- 代码生成器

- 输入：中间代码和符号表中的信息
- 输出：等价的目标程序



- 目标代码有以下三种：
 - **绝对机器代码**：能够立即执行的机器语言代码，所有地址均已定位；
 - **可再定位机器语言**：当需要执行时，由连接装入程序把它们和某些运行程序连接起来，转换成能执行的机器语言代码。
 - **汇编语言代码**：需经过汇编程序汇编，转换成可执行的机器语言代码。

例:

中间代码: $x := y + z$

目标代码:

LD R_0, y

ADD R_0, z

ST R_0, x

11.3 一个简单的代码生成器

- 11.3.1 待用信息
- 11.3.2 寄存器描述和地址描述
- 11.3.3 代码生成算法

11.3 一个简单的代码生成器

- 简单的代码生成器
 - 依次把每条中间代码变换成目标代码；
 - 在一个基本块范围内考虑如何充分利用寄存器。

11.3 一个简单的代码生成器

- 基本块(p279)

指程序中一顺序执行的语句序列，其中只有一个入口和一个出口，入口是其中的第一个语句，出口是其中的最后一个语句。

执行时，只能从其入口进入，从其出口退出。

例，基本块：

$t_1 := a * a$

$t_2 := a * b$

$t_3 := 2 * t_2$

$t_4 := t_1 + t_3$

$t_5 := b * b$

$t_6 := t_4 + t_5$

11.3 一个简单的代码生成器

- 基本块(p279)

如果一条三地址语句为 $x := y + z$ ，则称对 x 定值并引用 y 和 z 。

在一个基本块中的一个名字，所谓在程序中的某个给定点是活跃的，是指如果在程序中（包括在本基本块或在其它基本块中）它的值在该点以后被引用。

例，基本块：

$t_1 := a * a$

$t_2 := a * b$

$t_3 := 2 * t_2$

$t_4 := t_1 + t_3$

$t_5 := b * b$

$t_6 := t_4 + t_5$

11.3.1 待用信息

- 充分利用寄存器
 - 在一个基本块范围内考虑;
 - 把基本块内还要被引用的变量值尽可能保存在寄存器中;
 - 把基本块内不再被引用的变量所占用的寄存器及早释放;
 - 后续的目标代码尽可能地引用变量在寄存器中的值，而不访问主存。
- 如：翻译中间代码 $A := B \text{ op } C$
 - A、B、C是否还会在基本块内被引用？
 - 用于哪些中间代码中？

11.3.1 待用信息

- 待用信息

中间代码序列：

i: A := 10

⋮

j: B := A - C

中间代码i对A**定值**

从i到j之间没有A的其它定值

j**引用**了中间代码i中所计算的A值

j 是中间代码i 中 A 的待用信息。

11.3.1 待用信息

- 计算变量待用信息算法
 - 从基本块的出口由后向前扫描，对每个变量建立相应的待用信息链和活跃变量信息链。
 - 假设变量的符号表登记项中含有记录待用信息和活跃信息的栏（区段）。

11.3.1 待用信息

- 计算变量待用信息算法

(1) 基本块中各变量的符号表登记项中的待用信息栏填为“非待用”，根据该变量在基本块出口之后是不是活跃的，把其中的活跃信息栏填为“活跃”或“非活跃”。

(2) 从基本块出口到基本块入口依次处理各个中间代码。对每一中间代码 $i: A := B \text{ op } C$ ，依次执行下述步骤：

- ①把符号表中A的待用信息和活跃信息附加到中间代码i上；
- ②把符号表中A的待用信息和活跃信息分别置为“非待用”和“非活跃”；
- ③把符号表中B和C的待用信息和活跃信息附加到中间代码i上；
- ④把符号表中B和C的待用信息均置为i，活跃信息均置为“活跃”。

11.3.1 待用信息

例11.1 基本块

$T := A - B$

$U := A - C$

$V := T + U$

$W := V + U$

设 W 是基本块出口的活跃变量。

表11.1 符号表中的待用及活跃信息

变量名	下次引用信息及活跃信息
T	$(\wedge, \wedge) \rightarrow (3, Y) \rightarrow (\wedge, \wedge)$
A	$(\wedge, \wedge) \rightarrow (2, Y) \rightarrow (1, Y)$
B	$(\wedge, \wedge) \rightarrow (1, Y)$
C	$(\wedge, \wedge) \rightarrow (2, Y)$
U	$(\wedge, \wedge) \rightarrow (4, Y) \rightarrow (3, Y) \rightarrow (\wedge, \wedge)$
V	$(\wedge, \wedge) \rightarrow (4, Y) \rightarrow (\wedge, \wedge)$
W	$(\wedge, Y) \rightarrow (\wedge, \wedge)$

表11.2 附加在中间代码上的的待用及活跃信息

序号	中间代码	左值	左操作数	右操作数
1	$T := A - B$	$(3, Y)$	$(2, Y)$	$(\wedge, \wedge)$
2	$U := A - C$	$(3, Y)$	$(\wedge, \wedge)$	$(\wedge, \wedge)$
3	$V := T + U$	$(4, Y)$	$(\wedge, \wedge)$	$(4, Y)$
4	$W := V + U$	$(\wedge, Y)$	$(\wedge, \wedge)$	$(\wedge, \wedge)$

11.3.2 寄存器描述和地址描述

- 寄存器描述
 - 寄存器的情况：
 - 空闲；
 - 分配给某个变量；
 - 分配给某几个变量（复写，如 $A:=B$ ）
 - 寄存器描述数组RVALUE
 - 数组元素的下标：寄存器的编号值 R_i ；
 - 数组元素的值：变量名
 - 如： $RVALUE[R_i] = \{A, C\}$

11.3.2 寄存器描述和地址描述

- 地址描述

- 变量地址描述数组 **AVALUE**，动态地记录各变量现行值的存放位置：寄存器，或主存单元，或寄存器和主存单元。

- 如：

$AVALUE[A] = \{A\}$ A的值在内存中

$AVALUE[A] = \{R_i, A\}$ A的值在寄存器 R_i 中也在内存中

$AVALUE[A] = \{R_i\}$ A的值在寄存器 R_i 中

11.3.3 代码生成算法

- GETREG函数
 - **GETREG(i: A := B op C)** 给出一个用来存放A的值的寄存器R;
 - 要用到中间代码i上的待用信息。

11.3.3 代码生成算法

- GETREG($i: A := B \text{ op } C$)算法

- (1) 如果B的现行值在某寄存器 R_i 中，RVALUE[R_i]只包含B，此外，或者B与A是同一标识符，或者B的现行值在执行中间代码 $i: A := B \text{ op } C$ 之后不会再引用，则选取 R_i 为所需的寄存器R，并转4。
- (2) 如果有尚未分配的寄存器，则从中选取一个 R_i 为所需的寄存器R，并转4。
- (3) 从已分配的寄存器中选取一个 R_i 为所需的寄存器R。如果 R_i 保存的是变量w的值，并且该值还没有保存到w的内存单元，则生成ST R_i, w 指令，把 R_i 的值存入w，修改w的地址描述，返回R。如果 R_i 保存着几个变量的值，对于每个需要保存的变量都产生ST指令。
- (4) 给出R，返回。

11.3.3 代码生成算法

对每个中间代码 $i: A := B \text{ op } C$ ，依次执行下述步骤。

- (1) 以中间代码 $i: A := B \text{ op } C$ 为参数，调用函数过程 $\text{GETREG}(i: A := B \text{ op } C)$ ，得到寄存器 R ，用作存放 A 现行值的寄存器。
- (2) 利用地址描述数组 $\text{AVALUE}[B]$ 和 $\text{AVALUE}[C]$ ，确定变量 B 和 C 现行值的存放位置 B' 和 C' 。如果其现行值在寄存器中，则把寄存器取作 B' 和 C' 。
- (3) 如果 $B' \neq R$ ，则生成目标代码：
 LD R, B'
 op R, C'
 否则，生成目标代码：
 op R, C'
 如果 B' 为 R 且 A 和 B 不是同一变量名时，则删除 $\text{AVALUE}[B]$ 中的 R 。

11.3.3 代码生成算法

- (4) 令 $AVALUE[A]=\{R\}$ ，并令 $RVALUE[R]=\{A\}$ ，即变量A的现行值只在R中并且R中的值只代表A的现行值。
- (5) 如果B和C的现行值在基本块中不再被引用，它们也不是基本块出口之后的活跃变量，并且其现行值在某寄存器 R_k 中，则删除 $RVALUE[R_k]$ 中的B或C以及 $AVALUE[B]$ 或 $AVALUE[C]$ 中的 R_k ，使该寄存器不再被B或C所占用。

11.3.3 代码生成算法

例11.2 基本块

$T := A - B$

$U := A - C$

$V := T + U$

$W := V + U$

假设只有 R_0 和 R_1 是可用寄存器，用上述算法生成目标代码。

表11.2 附加在中间代码上的的待用及活跃信息

序号	中间代码	左值	左操作数	右操作数
1	T := A-B	(3, Y)	(2, Y)	($\wedge$, $\wedge$)
2	U := A-C	(3, Y)	($\wedge$, $\wedge$)	($\wedge$, $\wedge$)
3	V := T+U	(4, Y)	($\wedge$, $\wedge$)	(4, Y)
4	W := V+U	($\wedge$, Y)	($\wedge$, $\wedge$)	($\wedge$, $\wedge$)

表11.3 目标代码

中间代码	目标代码	RVALUE	AVALUE
T := A-B	LD R_0, A SUB R_0, B	R_0 含有T	T在 R_0 中
U := A-C	LD R_1, A SUB R_1, C	R_0 含有T R_1 含有U	T在 R_0 中 U在 R_1 中
V := T+U	ADD R_0, R_1	R_0 含有V R_1 含有U	V在 R_0 中 U在 R_1 中
W : = V+U	ADD R_0, R_1	R_0 含有W	W在 R_0 中

11.3.3 代码生成算法

处理完基本块中所有中间代码，对现行值只在某寄存器中的每个变量，如果它在基本块出口之后是活跃的，则要用ST指令把它在寄存器中的值存放到它的主存单元中。

表11.2 附加在中间代码上的的待用及活跃信息

序号	中间代码	左值	左操作数	右操作数
1	$T := A - B$	(3, Y)	(2, Y)	($\wedge$, $\wedge$)
2	$U := A - C$	(3, Y)	($\wedge$, $\wedge$)	($\wedge$, $\wedge$)
3	$V := T + U$	(4, Y)	($\wedge$, $\wedge$)	(4, Y)
4	$W := V + U$	($\wedge$, Y)	($\wedge$, $\wedge$)	($\wedge$, $\wedge$)

表11.3 目标代码

中间代码	目标代码	RVALUE	AVALUE
$T := A - B$	LD R_0, A SUB R_0, B	R_0 含有 T	T 在 R_0 中
$U := A - C$	LD R_1, A SUB R_1, C	R_0 含有 T R_1 含有 U	T 在 R_0 中 U 在 R_1 中
$V := T + U$	ADD R_0, R_1	R_0 含有 V R_1 含有 U	V 在 R_0 中 U 在 R_1 中
$W := V + U$	ADD R_0, R_1	R_0 含有 W	W 在 R_0 中

ST R_0, W